

GameController

这是用于 RoboCup 类人组足球联赛（RoboCup Humanoid Soccer League）机器人足球比赛的 GameController 软件。

编译 (Compilation)

先决条件 (Prerequisites)

- Rust 以及[此处列出的](#)其他特定于平台的 tauri 依赖项
- nodejs 和 npm（或兼容的包管理器）
- libclang（用于 bindgen）
 - 在 Windows 上：

```
winget install LLVM  
set LIBCLANG_PATH 'C:\Program Files\LLVM\bin\'
```

命令 (Commands)

首先，必须编译前端：

```
cd frontend  
npm ci  
npm run build
```

Rust 代码使用 cargo 编译：

```
cargo build [-r]
```

开发 (Dev)

首先，感谢您为开发做出贡献。

这将安装 tauri CLI：

```
cargo install tauri-cli
```

这将以开发模式启动前端和后端：

```
cargo tauri dev
```

创建二进制发行版 (Creating Binary Distributions)

对于 Linux、macOS 和 Windows，dist/ 目录下有用于创建二进制发行版的脚本。它们都期望将版本号作为第一个参数，并且可以选择性地将目标平台作为第二个参数（否则在 Linux 和 Windows 上默认为宿主目标，而在 macOS 上创建通用二进制文件）。它们构建一个名为 release-dist 的特殊配置文件（profile），主要旨在创建较小的二进制文件。建议仅在干净的工作树（clean working tree）上运行此脚本。

配置 (Configuration)

运行时读取的配置文件位于 config 目录下。全局的 teams.yaml 列出了 HSL 中的所有球队、它们的名称，以及场上球员和守门员可用的球衣颜色。每个（子）竞赛都有一个包含两个文件的子目录：params.yaml 对应于 Rust 结构体 game_controller_core::types::CompetitionParams，用于设置竞赛的（主要是数值）常量。teams.yaml 是参与该竞赛的球队编号列表。只有这些球队才能在选择该竞赛时被选中。因此，要让新球队出现在 UI 中，必须在全局 teams.yaml（使用未使用的球队编号）和竞赛的 teams.yaml（引用该球队编号）中都添加条目。

网络通信 (Network Communication)

目前，与 GameController 的所有网络通信都使用 IPv4，尽管代码的大部分部分也可以处理 IPv6。

GameController 通过三个通道与机器人球员通信：

- 它以 2 赫兹的频率发送控制消息（端口 3838 上的 UDP 广播，格式在 game_controller_msgs/headers/RoboCupGameControlData.h 中的 RoboCupGameControlData 结构体中指定）。这些控制消息并不总是代表真实的比赛状态，特别是在进球或转换到 playing（比赛进行中）状态之后。在这些事件之后，它们会继续保持事件发生前的状态长达 15 秒，或者直到发生了在该“伪造”状态下不可能发生的另一个事件。请注意，此行为与旧版 GameController 不同，旧版会始终将状态属性（以及其他一些属性）保持在旧值 15 秒，即使其他属性已经清楚地表明是新状态（例如，虽然计时器未归零，但球员已解除判罚，或者在实际已经是 playing 状态时，在“伪造”的 set 状态期间开始了定位球）。
- 它接收来自机器人球员的状态消息，机器人必须以 0.5 赫兹到 2 赫兹的频率发送这些消息（端口 3939 上的 UDP 单播，格式在 game_controller_msgs/headers/RoboCupGameControlData.h 中的 RoboCupGameControlReturnData 结构体中指定）。
- 它接收来自机器人球员的团队消息（端口 10000 + 球队编号上的 UDP 广播，最大 512 字节的任意格式有效载荷）。

此外，GameController 为监控应用程序（如 TeamCommunicationMonitor 或 EventRecorder）提供接口：

- 它接收监控请求（端口 3636 上的 UDP 单播，4 字节头部魔数 RGT + 1 字节版本号 0）。它拒绝接受来自之前已发送过状态消息的主机的监控请求，因为这些主机被假定为不应获取真实数据的机器人球员。同样，如果之前发送过监控请求的主机发送了状态消息，它将不再接收监控数据。
- 每个注册的监控主机将获得：

- 包含真实比赛状态的控制消息，频率为 2 赫兹（端口 3838 上的 UDP 单播，格式与常规控制消息相同，但头部魔数为 RGTD）。
- 转发的状态消息（端口 3940 上的 UDP 单播，前缀为原始发送者的 IPv4 地址）。转发的有效载荷未经验证。

用户必须确保防火墙允许使用上述所有网络通信通道。GameController 在特定的网络接口上运行，该接口通常指定了数据包的发送目的地和接收来源。例外情况是，控制消息可以配置为发送到受限广播地址（255.255.255.255）而不是接口的广播地址，并且监控请求和团队消息可以从任何地址接收。

用法 (Usage)

启动 (Start)

[GitHub 发布页面](#)上的二进制发行版附带了可以用平台典型方式执行的脚本。在 macOS 上，您可能需要在首次运行之前调用 `xattr -c <GameController.app 的路径>` 来清除隔离标志。

如果应该从源代码运行 GameController，最方便的方法是在此工作区的任何目录下的命令行中执行：

```
cargo run [-r]
```

该程序接受可以在 `--` 之后传递给 cargo 的命令行参数。它们会覆盖启动器的默认设置。通过使用 `-h` 选项运行，可以获得参数列表（与此处编写的内容相比，该列表始终是最新的）：

```
cargo run -- -h
```

请注意，Windows 上的发布版本（release builds）不会输出任何文本。

启动器 (Launcher)

当 GameController 启动时，首先会显示一个启动器。如果指定了命令行参数，某些字段将被预先填充。

通过启动器可以进行以下设置：

- **Competition (竞赛)**：此框选择比赛的竞赛类型。它会影响 GameController 的行为和常量，并缩小可选择的球队范围。
- **Teams (球队)**：
 - **Kick-off for (home / away) team (主队/客队开球)**：此框选择哪支球队拥有第一次开球权，这是赛前抛硬币的结果。
 - 主选框选择各侧的球队。
 - **Field player color (场上球员颜色)** 和 **goalkeeper color (守门员颜色)**：这些框选择球队的球衣颜色。
- **Testing (测试)**：
 - **No Delay (无延迟)**：此复选框禁用比赛状态转换的延迟。

- **Penalty Shoot-out (点球大战)**：此复选框允许开始/继续点球大战，即使比赛实际上已经分出胜负。
- **Unpenalize (解除判罚)**：此复选框允许在时间结束前解除对球员的判罚。
- **Mirror (镜像)**：此复选框选择主队（赛程表上的第一支球队）是否防守右侧（从 GameController 的视角），而不是左侧，这是赛前抛硬币的结果。
- **Fullscreen (全屏)**：此复选框选择启动时窗口是否应切换到全屏模式。
- **Interface (接口)**：此框选择运行的网络接口（见[上文](#)）。并非所有列出的接口都能正常工作。
- **Broadcast (广播)**：此复选框选择是否将控制消息发送到受限广播地址（255.255.255.255）而不是接口的广播地址。仅当要求在所有接口上发送这些消息时才应使用此选项。
- **Multicast (多播)**：此复选框选择是否也从特定的多播组接收团队通信。这应该仅用于模拟比赛，绝不适用于真实的竞赛比赛。

只有当两支球队不同且球衣颜色不冲突时（即所有四种颜色必须两两不同，除了两队的守门员颜色可以相同外），启动器才允许开始比赛。请注意，之后无法更改场地侧别或开球球队，因此切换到主界面只能在抛硬币之后进行。

主界面 (Main Interface)

球队面板（即布局的左列和右列）应与球队防守的球门对齐（从 GameController 操作员的视角来看）。也就是说，GameController 操作员的视图定义了什么是左侧和右侧球队，正如在[启动器](#)中选择的那样。

主界面有时是上下文敏感的，即某些按钮仅在当前比赛状态下有用时才会出现。但是，通常会避免在没有用户交互的情况下更改按钮的布局或含义（因为如果用户正要按下的按钮在那一瞬间改变了含义，那将是很糟糕的）。

判罚 (Penalties)

实施判罚的方法是：先点击标有判罚名称的按钮，然后再点击被判罚球员的按钮。这将在相应球员的按钮上启动一个计时器，一旦剩余时间只有 10 秒，计时器就会开始闪烁。球员不会自动解除判罚。相反，一旦判罚时间结束并且助理裁判已将他们正确安置，必须通过点击他们的按钮来解除判罚。如果错误地判罚了球员，并且随后的操作导致无法简单地[撤销](#)，可以通过按住 *Shift* 键的同时点击该球员来提前移除判罚。但是，这不会移除对球队判罚计数器的影响，也不会移除对后来被判罚的其他球员的判罚时间的影响。

有些判罚是特殊的：

- **Pick-Up (提取)** 判罚可以替换任何现有的判罚，尽管计时器会继承自原始判罚。
- **Motion in Standby/Set (Standby/Set 状态下移动)** 判罚可以应用于多名球员而无需重复选择判罚类型，以适应多名球员同时对错误的哨声做出反应的情况。此外，当计时器结束后，球员会自动解除判罚，因为他们从未被移出场地。

消息计数 (Message Counting)

根据规则书，在 Ready、Set 和 Playing 状态期间（但不在点球大战中），会对团队消息进行计数（基于发送消息的端口）。当收到的团队消息过大或超出团队的消息预算时，得分将重置为 0，并且未来的进球将不被计入。此时得分和消息计数器会变为不同的颜色。

暂停 (Timeouts)

球队和裁判都可以请求暂停。当在 Ready 或 Set 状态期间进行暂停时，时钟会重置到比赛停止的开始时刻，但消息预算不会重置。

当在半场休息计时器倒计时期间进行暂停时，计时器会根据暂停的持续时间进行调整。在所谓的交错比赛（interleaved games）中，这可能会引起困惑，因为半场休息时间通常比平时长，因此即使在暂停后，半场休息计时器仍可能为负数。

加时 (Extra Time)

通过按主时钟旁边的“+”按钮可以增加一分钟的加时。一旦半场比赛进行了一分钟，此按钮就会在比赛停止期间变为可用。增加一分钟的加时也会增加两队的消息预算，除非某队此前曾进行过非法通信。

换人 (Substitution)

要替换球员，首先点击球队一侧的“Substitute (换人)”按钮。然后，点击应从比赛中移除的球员。球员列表现在将变为可用替补球员的列表（请注意，此列表可滚动）。从列表中点击要替换先前所选球员的球员。根据比赛状态，新球员将获得判罚或继承被替换球员的判罚。守门员属性会转移给替补球员，即当守门员被替换时，替补球员应穿着守门员球衣并继承守门员的特权。

此功能也必须在半场开始前使用，以便将 GameController 中的球员集与实际在场上的球员相匹配。如果一支球队想要使用号码为 2-7（或挑战盾杯中的 2-5）的球员作为守门员，这必须通过三次换人来完成（例如，如果一支球队想使用球员 1-7，但让 3 号作为守门员，它必须用 8 号换下 1 号，用 1 号换下 3 号，用 3 号换下 8 号）。

点球大战 (Penalty Shoot-out)

只有在两个半场比赛结束且比分平局后，才能开始点球大战。但是，这并不取决于比赛模式，因为在 Ready 和 Set 状态期间时钟不停止的比赛中也可能需要点球大战。只要比赛结果尚不明确，就允许切换到下一次射门。

一旦双方都选择了球员，就可以进入 Playing 状态。这是通过点击“Select (选择)”按钮（代替“Substitute”按钮）来完成的。首先，必须选择所选球员的颜色。这是因为在点球大战中，球衣颜色不一定与守门员或踢球者的指定相匹配，GameController 无法知道哪些球员穿着哪种球衣。选择颜色后，会出现一个可滚动的球员列表（类似于换人）。

撤销 (Undo)

在窗口底部，有一个时间轴，从右到左填充了用户最近执行的五个操作。点击那里的某个操作会将整个比赛恢复到该操作应用之前的状态。无法撤销已经被其他操作跟随的单个操作。撤销历史实际上并不限于最后五个操作，即一旦撤销了一些操作，之前的操作就会出现。

自动应用的操作（例如因为计时器结束）不会出现在撤销历史记录中。这是因为如果撤销它们，它们会立即再次被应用。

日志 (Logs)

GameController 将日志文件写入 logs 目录。它们可能会变得很大，因为它们是 YAML 格式。使用 YAML 的主要原因是它是人类可读的，并且可以追加（与 JSON 不同，JSON 需要在末尾加上闭合括号才能格式良好）。

这些日志文件可以通过 game_controller_logs 包中的程序进行分析，或使用 [TeamCommunicationMonitor](#) 进行重播。